

Identifying Coordinated Amazon Review Groups with Distributed Text and Graph Analysis

CSC 7740 — Big Data Analytics
Final Project Report

Submission Date: **[TODO: submission date]**

Team Members

Souhardya Saha Dip	ssahad1@lsu.edu
Riyadil Zannat	rzanna2@lsu.edu
Asif Faisal Chowdhury	achowd6@lsu.edu

Prior CSC 4740 Enrollment

No member of this team has previously taken CSC 4740 at LSU.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Project Description	1
1.3	Division of Roles	2
2	System Architecture and Design	2
2.1	Chosen Big Data Frameworks	2
2.2	Chosen Datasets	3
3	Detailed Component Description	5
3.1	load_enriched — Ingest, Join, Feature Derivation	5
3.2	add_group_key — Adaptive Time Bucketing	5
3.3	repeated_pairs — Co-occurrence Self-Join	5
3.4	pair_audit and drop_same_account_pairs	5
3.5	connected_components and label_propagation	6
3.6	text_similarity — MinHash LSH	6
3.7	component_stats and score — Seven Signals	6
3.8	suspicious_reviews	7
4	Evaluation, Testing, and Execution	7
4.1	Evaluation Environment	7
4.2	Execution Instructions (Software Manual)	8
4.3	Results and Findings	9
4.3.1	Corpus-scale detection results	9
4.3.2	Parameter sensitivity	10
4.3.3	Validation by synthetic injection	10
4.3.4	Cluster scale-out: one worker versus two	11
4.4	System Screenshots	12
5	Conclusion	13
A	Appendix 1: Source Code Summary and Engineering Contribution	14

1 Introduction

1.1 Motivation

Product reviews are the main signal shoppers use when they cannot inspect an item directly, and that makes them worth manipulating. The form of manipulation that matters at scale is not the single fake review but the *coordinated group*: a set of accounts that move together, reviewing the same products inside the same narrow time windows, often with similar text and similar ratings. A single review in isolation looks unremarkable. The coordination only becomes visible when many accounts are compared against each other across the entire corpus.

That comparison is inherently a big-data problem. Deciding whether two accounts co-occur repeatedly requires grouping every review by product and time window, then examining every pair of accounts inside every group. On the four Amazon categories we study, that means 24.4 million reviews collapsing into 20.6 million candidate product-time groups, from which account pairs must be generated and counted. The pair-generation step alone is quadratic in group size, so the work cannot be done by reading the data once on a single machine with a hash map; it needs a shuffle-based distributed engine that can spill to disk and parallelise the self-join.

The practical significance is twofold. For a platform, a ranked list of suspicious account groups is a triage queue for human moderators. For researchers, the same pipeline is a measurement instrument: it quantifies how much apparent coordination exists in a public corpus, and how sensitive that quantity is to the thresholds an analyst chooses.

1.2 Project Description

We built a distributed Spark pipeline that ingests the raw Amazon Reviews’23 corpus, enriches it with product metadata, constructs an account co-review graph, extracts connected components as candidate coordinated groups, and ranks those groups by a seven-signal coordination score.

The pipeline has five stages:

1. **Ingest and enrich.** Read gzip-compressed JSONL reviews under an explicit schema, join to product metadata on `parent_asin`, and derive per-review features including deviation from the product’s catalogue average rating.
2. **Candidate grouping.** Bucket each review into a (product, time-window) key. Windows are 24 hours by default, narrowed to 6 hours for products with more than 100 reviews in a window so that popular items do not generate spurious co-occurrence.
3. **Graph construction.** Self-join accounts within each group to produce co-occurring pairs, keep only pairs appearing together in at least three distinct groups, and audit the survivors for same-underlying-account artefacts.
4. **Community detection.** Run distributed connected components over the pair graph using GraphFrames, with a Pregel-style label propagation implementation as a dependency-free fallback.

5. **Scoring and output.** Compute per-component statistics, including MinHash LSH text similarity, combine them into a single coordination score, and write ranked results to Parquet.

Beyond the detector itself, we treated the project as a measurement study. We ran a parameter sensitivity sweep across time windows and co-occurrence thresholds, validated recall against synthetically planted coordination rings, and measured scale-out behaviour by executing the identical pipeline on a one-worker and a two-worker AWS EMR cluster.

1.3 Division of Roles

TODO — action required before submission.

The team must confirm and correct this table before submission. It is filled in from the repository's commit history and the division of work as executed, but each member should verify their own row.

Member	Contributions and Responsibilities
Souhardya Saha Dip	Pipeline architecture and the DataFrame implementation (<code>AmazonReviewCoordinationDF.py</code>); schema definitions and the metadata join; the seven-signal scoring function; GraphFrames integration and the label-propagation fallback; same-account pair auditing; AWS EMR cluster provisioning, S3 data staging, and the one-worker versus two-worker scale-out experiment; parameter sensitivity sweep and synthetic injection validation; performance profiling and this report.
Riyadil Zannat	[TODO: confirm] Repository setup and the containerised development environment (<code>Dockerfile</code> , <code>.devcontainer</code>); initial pipeline prototype and per-category run scripts (<code>run_all_categories.sh</code> , <code>run_local.sh</code>); dataset acquisition scripting.
Asif Faisal Chowdhury	[TODO: confirm] Exploratory analysis scripts (<code>analysis/feasibility.py</code> , <code>analysis/pairest.py</code> , <code>analysis/lockstep.py</code> , <code>analysis/nearDup.py</code>) used to size the problem and choose the co-occurrence threshold; sample dataset construction for the deliverable package.

2 System Architecture and Design

2.1 Chosen Big Data Frameworks

Apache Spark (PySpark DataFrame API), version 3.5. Spark is the right engine here because the dominant cost is a *shuffle*, not a scan. Pair generation requires regrouping 24.4 million reviews by a synthetic (product, time-window) key and then self-joining within each group; community

detection requires iterative message passing over the resulting graph. Both are shuffle-bound operations that Spark’s execution model handles natively, with spill-to-disk when a partition exceeds memory.

We deliberately used the **DataFrame** API rather than RDDs. An earlier RDD implementation of this pipeline failed repeatedly with out-of-memory errors, which we traced to the JVM-to-Python serialisation boundary: every row crossing into a Python lambda is pickled, and the `stdout` writer for `python` thread accumulated faster than it drained. Rewriting the same logic as DataFrame expressions keeps the data and the computation inside the JVM, where Catalyst can also push down projections and plan the joins. This change alone turned a pipeline that could not complete into one that processes the full corpus. It is the single most consequential engineering decision in the project.

Spark MLlib — MinHashLSH. Measuring review-text similarity within a group is an all-pairs problem, and computing exact Jaccard similarity across every pair of reviews in a component is not tractable at this scale. MinHash locality-sensitive hashing approximates it by hashing shingled token sets into bands, so that only candidate pairs that collide in a band are compared. MLlib’s MinHashLSH with `approxSimilarityJoin` gives this as a distributed operation over a DataFrame of sparse vectors.

GraphFrames 0.8.4. Connected components on the account graph is an iterative graph computation. GraphFrames provides a distributed implementation backed by GraphX, invoked with `algorithm="graphx"` and a checkpoint directory. Because GraphFrames is an external package resolved from the Spark Packages repository at submit time, we also implemented a Pregel-style label propagation fallback in pure DataFrame operations, selectable with `USE_GRAPHFRAMES=0`, so the pipeline runs in an offline or dependency-restricted environment.

AWS EMR on EC2, Amazon S3, and Apache Parquet. EMR gave us a real multi-node YARN cluster whose size we could change between otherwise identical runs, which is exactly what the scale-out measurement requires. S3 holds both the input corpus and the results, decoupling storage from the ephemeral cluster, and Parquet is the output format because the results are read back column-wise for analysis.

2.2 Chosen Datasets

We use the **Amazon Reviews’23** corpus released by the McAuley Lab at UC San Diego (<https://amazon-reviews-2023.github.io/>). We selected four categories that are large enough to be a genuine distributed workload while spanning different product types and review cultures.

Component	Contents	Compressed	Raw
User reviews	24,447,530 reviews across Video Games, CDs & Vinyl, Arts Crafts & Sewing, and Baby Products; 1996–2023	3.71 GB	12.89 GB
Item metadata	1,858,398 products with store/brand, price, category hierarchy, catalogue average rating and rating count	1.05 GB	4.25 GB
Total	gzip-compressed JSONL (1 GB = 10^9 bytes)	4.76 GB	17.14 GB

We verified these figures by decompressing and counting every file; the per-file line counts sum to exactly the 24,447,530 reviews and 1,858,398 products the pipeline reports, which confirms that no records are lost in ingest.

Category	Reviews	Products	Raw bytes
Arts, Crafts & Sewing	8,966,758	801,446	6.12 GB
Baby Products	6,028,884	217,724	3.64 GB
CDs & Vinyl	4,827,273	701,959	4.24 GB
Video Games	4,624,615	137,269	3.12 GB
Total	24,447,530	1,858,398	17.14 GB

Structure. Each review record is one JSON object per line with `user_id`, `asin`, `parent_asin`, `rating`, `timestamp` (milliseconds since epoch), `text`, `helpful_vote`, and `verified_purchase`. Metadata records carry `parent_asin`, `main_category`, `title`, `average_rating`, `rating_number`, `price`, and `store`.

Pre-processing pipeline. We read both sources under *explicit* `StructType` schemas rather than letting Spark infer them. This was not a cosmetic choice: schema inference requires a full extra pass over the data, and on the metadata files it failed outright with a `COLUMN_ALREADY_EXISTS` error, because the free-form `details` object contains keys that differ only in case (for example `Assembly Required` and `assembly required`). Declaring only the eight review fields and six metadata fields we actually use avoids both problems and reduces the bytes deserialised.

Reviews are then filtered for a non-null `user_id`, `parent_asin` and `timestamp`; metadata is deduplicated on `parent_asin`; and the two are joined. A review’s `rating_deviation` is computed as its rating minus the product’s catalogue average. Reviews shorter than 30 characters are excluded *only* from the MinHash text analysis, not from the corpus, because a short review still carries valid timing and rating evidence.

One property of the input shaped the whole performance story: the corpus arrives as four gzip files, and **gzip is not splittable**. A gzip member must be decompressed sequentially from its start, so Spark cannot divide one file across tasks. The ingest stage is therefore capped at four parallel tasks regardless of cluster size, which becomes important in Section 4.3.4.

3 Detailed Component Description

All logic lives in `AmazonReviewCoordinationDF.py` (612 lines). Every tunable is an environment variable with a default, so the same file runs locally and on EMR without modification.

3.1 `load_enriched` — Ingest, Join, Feature Derivation

Reads reviews and metadata under the explicit schemas, filters nulls, deduplicates metadata on `parent_asin`, and performs the enrichment join. Derives `ts` (seconds), `year`, `review_text` (null-coalesced), `helpful_votes`, `verified`, `category`, `product_title`, and `rating_deviation`. The join is a standard shuffle hash join on `parent_asin`; metadata is the smaller side.

3.2 `add_group_key` — Adaptive Time Bucketing

Assigns each review a `group_key` of the form `parent_asin # bucket`. The bucket is $\lfloor ts / (3600 \cdot 24) \rfloor$ by default. A fixed window is wrong for popular products: a best-selling game can draw hundreds of unrelated reviews in a day, and every pair of those reviewers would look like a co-occurrence. The component therefore counts reviews per (product, 24h-window) first, and for any window exceeding `LARGE_GROUP_THRESHOLD = 100` reviews, re-buckets that window at `LARGE_GROUP_BUCKET_HOURS = 6` hours. This is a single `groupBy/agg` followed by a broadcastable join and a `when` expression, so the refinement costs one extra shuffle rather than a per-product loop.

3.3 `repeated_pairs` — Co-occurrence Self-Join

The quadratic core. Distinct (`group_key`, `user_id`) rows are self-joined on `group_key` with the predicate `user1 < user2`, which both removes self-pairs and canonicalises ordering so each unordered pair appears once. The result is grouped by (`user1`, `user2`) and counted, and only pairs with `repeated_groups` $\geq$ `MIN_REPEATED_GROUPS = 3` survive.

Requiring three distinct co-occurrences, rather than one, is what makes the output tractable and meaningful: two strangers can easily review the same product on the same day once, but doing so three separate times is a strong signal. Section 4.3.2 quantifies how much this threshold matters.

3.4 `pair_audit` and `drop_same_account_pairs`

A data-quality guard we added after inspecting early results. The corpus contains user IDs that are the same base identifier with a numeric suffix (for example `AHPG65LK...` and `AHPG65LK..._1`). These produced the highest co-occurrence counts in the graph — one pair co-occurred 464 times — but they almost certainly represent one underlying account, not two coordinating ones. `pair_audit` flags pairs whose IDs share a base after suffix stripping, writes

them to a CSV audit trail, and `drop_same_account_pairs` removes them before graph construction. Keeping this as an explicit, auditable step rather than a silent filter means the decision is visible and reversible.

3.5 `connected_components` and `label_propagation`

Builds a `GraphFrame` from the surviving pairs and runs `connectedComponents(algorithm="graphx")` against a configured checkpoint directory. Each resulting component is one candidate coordinated group. `label_propagation` is the fallback: it initialises each vertex with its own ID, then iteratively propagates the minimum neighbour label for up to 20 iterations, terminating early when no label changes. It is written entirely in `DataFrame` operations with `localCheckpoint` to truncate the lineage each round, which is necessary because iterative `DataFrame` computations otherwise build a query plan that grows without bound.

3.6 `text_similarity` — MinHash LSH

Restricted to reviews belonging to graph components, capped at `MAX_REVIEWS_PER_COMPONENT = 500` per component to bound the all-pairs cost. Review text is lowercased, tokenised on non-word characters, filtered to tokens of length ≥ 2 , and converted to a term-frequency vector via `CountVectorizer`. `MinHashLSH` with `MINHASH_NUM_HASH_TABLES = 5` and a fixed `MINHASH_SEED = 42` produces hash signatures, and `approxSimilarityJoin` returns candidate pairs within a Jaccard distance corresponding to `TEXT_SIMILARITY_THRESHOLD = 0.70`. Pairs are filtered to those whose two reviews sit in the same component and belong to different accounts. The seed is fixed deliberately so that runs are reproducible; `MinHash` is randomised, and without a fixed seed the similarity counts would differ between the one-worker and two-worker runs and contaminate the comparison.

3.7 `component_stats` and `score` — Seven Signals

`component_stats` aggregates per component: user count, review count, mean rating, mean absolute rating deviation from catalogue average, verified rate, mean helpful votes, busiest single hour, edge count, mean repetition, time concentration (busiest hour / total reviews), rating agreement ($1 - \text{MAD}/4$), and graph density ($\text{edges} / \binom{\text{users}}{2}$).

`score` clamps each signal to $[0, 1]$ and combines them:

Signal	Weight
Group size (normalised by 20 accounts)	0.18
Graph density	0.18
Repeated co-review frequency (normalised by 10)	0.18
Text similarity (MinHash)	0.14
Time concentration	0.13
Deviation from catalogue rating (normalised by 2 stars)	0.10
Rating agreement	0.09
Total	1.00

The weights are asserted to sum to 1.0 at import time. They encode a judgement that structural evidence (size, density, repetition) is more trustworthy than behavioural evidence (rating agreement), because the latter is easy to produce by accident. We make no claim that these weights are optimal; they are a documented, reproducible prior, and Section 4.3.2 reports how the structural thresholds behind them affect the output.

3.8 suspicious_reviews

Joins the enriched corpus back to the top-scoring components and writes the individual reviews belonging to them, so that a human reviewer can inspect the evidence behind a ranked group rather than only its score.

4 Evaluation, Testing, and Execution

4.1 Evaluation Environment

Development and baseline (single node). Linux workstation, PySpark 3.5 in local mode with `local[4]` and an 8 GB driver heap. Used for the full-corpus baseline, the sensitivity sweep, and the injection test.

Cluster (AWS EMR on EC2).

Release label	emr-7.14.0
Applications	Hadoop 3.4.2, Spark 3.5.8-amzn-0, JDK 17
Cluster ID	j-05114253MUVDJMJS0Y5
Primary node	1 × m5.xlarge (4 vCPU, 16 GB)
Core nodes	1 × m5.xlarge (Run A); 2 × m5.xlarge (Run B)
Deploy mode	YARN, client mode
Driver memory	4 GB
Shuffle partitions	400 (identical in both runs)
Storage	s3://csc7740-amazon-review/
External package	graphframes:graphframes:0.8.4-spark3.5-s_2.12
Cluster lifetime	7.78 hours, auto-terminated on a 1-hour idle policy

Three EMR configuration problems were resolved during bring-up and are worth recording because each produced a failure that looked unrelated to its cause. All three traced to a single root: the application code set `spark.hadoop.fs.defaultFS` to `file:///` in the `SparkSession` builder. Builder options are applied on top of `spark-submit` options when the context is created in client mode, so this silently overrode the cluster's real `fs.defaultFS` and caused every scheme-less or host-less path to resolve to the primary node's local disk. The symptoms were: the `ApplicationMaster` on a core node being unable to read a staging directory written to local disk; an `Incomplete HDFS URI`, no host failure on EMR's default `hdfs:///var/log/spark/apps event-log path`; and a `FileNotFoundException` from `RawLocalFileSystem` when `setCheckpointDir` resolved a bare path. The fix was to apply `file:///` only when the master URL begins with `local`.

4.2 Execution Instructions (Software Manual)

1. Prerequisites. Python 3.9+, Java 17, and `pip install -r requirements.txt` (PySpark 3.5, NumPy). `GraphFrames` is resolved at submit time and needs network access; set `USE_GRAPHFRAMES=0` to use the built-in fallback instead.

2. Obtain the data.

```
bash analysis/download_data.sh          # fetches 4 review + 4 metadata .jsonl.gz
```

3. Run locally.

```
bash run_local.sh
# or, explicitly:
REVIEWS_PATH='data/reviews/*.jsonl.gz' \
METADATA_PATH='data/meta/*.jsonl.gz' \
OUTPUT_ROOT=output/df_full \
SPARK_MASTER='local[4]' DRIVER_MEMORY=8g SHUFFLE_PARTITIONS=200 \
python3 AmazonReviewCoordinationDF.py
```

4. Run one category at a time, useful on a memory-constrained machine:

```
bash run_all_categories.sh
```

5. Deploy to AWS EMR.

```
# a) stage data and code
aws s3 cp data/reviews/ s3://<bucket>/data/reviews/ --recursive
aws s3 cp data/meta/    s3://<bucket>/data/meta/      --recursive
aws s3 cp AmazonReviewCoordinationDF.py s3://<bucket>/project/
aws s3 cp emr/run_emr.sh                s3://<bucket>/project/

# b) create the cluster (emr-7.14.0, Hadoop + Spark, m5.xlarge)
#     with a bootstrap action pointing at emr/bootstrap.sh

# c) submit the pipeline as a step
aws emr add-steps --cluster-id <id> --steps file:///emr/step.json
```

`emr/run_emr.sh` is the step entry point. It exports `HADOOP_CONF_DIR`, queries HDFS for the real `fs.defaultFS`, pins the staging directory, event-log directory, SQL warehouse and checkpoint directory to fully-qualified `hdfs://host:port` URIs, creates the checkpoint directory up front, and then calls `spark-submit`.

6. Reproduce the experiments.

```
python3 analysis/sensitivity.py    # window x threshold sweep
python3 analysis/injection.py     # synthetic ring recall test
python3 analysis/count_spark_operations.py  # Appendix 2 counts
```

Environment variables. Every tunable is overridable without editing code:

REVIEWS_PATH	METADATA_PATH	OUTPUT_ROOT	SPARK_MASTER
DRIVER_MEMORY	SHUFFLE_PARTITIONS		CHECKPOINT_DIR
USE_GRAPHFRAMES		GRAPHFRAMES_PACKAGE	
MINHASH_SEED	MAX_REVIEWS_PER_COMPONENT		WRITE_ENRICHED

4.3 Results and Findings

4.3.1 Corpus-scale detection results

The full pipeline run on all 24.4 million reviews produced identical results on the single-node baseline and on both EMR cluster configurations:

Quantity	Value
Reviews ingested and enriched	24,447,530
Candidate product-time groups	20,640,171
Co-occurring account pairs (≥ 3 groups)	1,051
Pairs flagged as same-account artefacts	28
Pairs dropped before graph construction	13
Edges in final graph	1,038
Graph vertices (accounts)	1,294
Connected components	443

That three separate executions on different hardware and different cluster topologies agree exactly on all eight figures is the strongest correctness evidence we have: the pipeline is deterministic, and the fixed MinHash seed holds.

Interpretation. 1,294 accounts out of millions, grouped into 443 components, is a small fraction — and that is the honest headline. Most components are account *pairs*: of the top 20 by score, 17 have exactly two members. The highest-scoring group (component 1151051235330, score 0.649) is two accounts that co-reviewed 11 times with perfect text similarity across 64 reviews. The structurally largest components are far bigger — one has 223 accounts, 331 edges and 33,694 reviews — but score lower (0.504) precisely because density falls as a component grows, which is the intended behaviour of the density signal.

4.3.2 Parameter sensitivity

Because the output size depends on two analyst choices — the time window and the co-occurrence threshold — we swept both on the Video Games category (4,624,615 reviews). The table reports surviving pairs, accounts, and components, plus the largest component found.

Window	Groups	Thresh.	Pairs	Accounts	Comps.	Largest
6h	3,997,723	2	1,532	2,376	1,051	30
		3	58	95	44	4
		5	1	2	1	2
		10	0	0	0	0
12h	3,751,135	2	2,804	4,229	1,781	230
		3	110	184	82	7
		5	2	4	2	2
		10	1	2	1	2
24h	3,428,395	2	4,989	7,037	2,784	675
		3	224	338	129	34
		5	10	16	7	4
		10	1	2	1	2
72h	2,756,660	2	13,519	16,181	5,432	2,189
		3	637	873	310	177
		5	34	47	16	9
		10	3	6	3	2
168h	2,203,416	2	25,879	26,644	7,786	5,524
		3	1,302	1,603	515	328
		5	97	107	26	40
		10	4	8	4	2

Two findings matter. First, the threshold is far more powerful than the window: at a fixed 24-hour window, moving from 2 to 3 co-occurrences cuts surviving pairs by 95% (4,989 $\rightarrow$ 224). Second, widening the window inflates results superlinearly — 168 hours at threshold 3 yields 5.8 $\times$ the pairs of 24 hours — and produces a 328-account component that is almost certainly an artefact of popular products rather than coordination. Our defaults (24h, threshold 3) sit at the knee of both curves. Anyone reporting a count from a detector like this must publish its thresholds; the same corpus supports answers spanning three orders of magnitude.

4.3.3 Validation by synthetic injection

Sensitivity tells us how the output moves, not whether it is right. To measure recall we planted six synthetic coordination rings (125 reviews) into the 4,624,615-review Video Games corpus and re-ran detection at the default settings.

Ring	Accts	Found	Recall	Foreign	Expected
R1 tight, 3 accounts, 5 products	3	3	100%	0	detect
R2 tight, 5 accounts, 3 products	5	5	100%	0	detect
R3 large, 10 accounts, 3 products	10	10	100%	0	detect
R4 below threshold, 2 products	5	0	0%	0	miss (2 < 3)
R5 staggered beyond window	5	0	0%	0	miss (outside 24h)
R6 partial, 8 accounts, 6 products	8	7	88%	0	partial

Every ring designed to be detectable was recovered completely, the two designed to fall outside the stated operating envelope were correctly missed, and — importantly — **zero foreign accounts** were pulled into any planted ring’s component. The detector does not merge unrelated accounts into synthetic groups. The two deliberate misses are not defects but a measured statement of the envelope: coordination spread across only two products, or staggered beyond 24 hours, is invisible at the default settings and would require lowering the threshold or widening the window, with the false-positive cost quantified in the sensitivity table.

4.3.4 Cluster scale-out: one worker versus two

Both runs executed the identical code, commit, and settings; only the core node count differed. Run A used one core node, Run B two.

Stage	1 worker	2 workers	Speedup
enrich + join	80.9 s	64.8 s	1.25×
candidate groups	178.2 s	105.4 s	1.69×
repeated pairs (self-join)	40.8 s	26.1 s	1.56×
pair audit	1.9 s	1.6 s	1.19×
connected components	6.4 s	5.6 s	1.14×
MinHash text similarity	276.1 s	175.8 s	1.57×
component stats + score	6,381.2 s	5,359.3 s	1.19×
write results	91.1 s	56.8 s	1.60×
Total (wall clock)	7,387 s	6,046 s	1.22×

Doubling the workers gave only a **1.22×** overall speedup. The reason is visible in the table: every stage except one scaled respectably (1.14–1.69×), but **component stats + score** consumed **86.6%** of Run A and improved by only 1.19×, pinning the overall figure to itself.

We profiled that stage rather than guessing. Three independent sources agree:

- **CloudWatch.** The core node sat at a flat 27% CPU — almost exactly one of four vCPUs — for over an hour, with EBS writes of roughly 20 KB/s. The work was CPU-bound on a single thread at a time, not I/O-bound.

- **YARN metrics.** `ContainerAllocated` was 2 (one `ApplicationMaster`, one executor) with `YARNMemoryAvailablePercentage` at 1.04% and `ContainerPending` around 12. Spark was requesting roughly six times the executors YARN could grant: Run A was resource-starved, not merely slow. After the resize, allocation rose to 3 containers and available memory to 39–50%, confirming the second node was used.
- **Executor logs.** Stage 1482 completed 56 tasks between 20:10:57 and 20:50:23, about 1.4 tasks per minute, each reading roughly 12 MB of shuffle. Every fetch logged `Started 0 remote fetches`, i.e. all shuffle reads were local to the single node.

Root cause. In `component_stats`, the intermediate `member_reviews = enriched.join(components, "user_id")` is consumed four times — by the per-component aggregation, the busiest-hour aggregation, the rating-agreement aggregation, and alongside edge statistics — and is never cached. Neither is `enriched`. Spark therefore recomputes the entire ingest-and-join pipeline for each consumer, re-reading and re-parsing 3.7 GB of *non-splittable* gzip every time. Because gzip caps that re-read at four tasks, and Run A had only two executor cores, the stage degenerated to roughly one core of useful work.

Why the second worker helped so little. Run B doubled task slots from two to four, which should have halved a 139-task stage. It did not, and the likely reason is the mirror image of the evidence above: with one executor, Run A fetched every shuffle block locally; with two executors on separate nodes, Run B must fetch roughly half of them across the network. The added parallelism and the added network cost largely cancelled within this stage. We state this as the probable mechanism rather than a confirmed one — distinguishing it conclusively would require comparing remote-fetch counts in both runs’ executor logs, which we have retained but not yet analysed.

The fix, and why we did not apply it mid-experiment. Persisting `member_reviews` is a one-line change projected to cut total runtime by 3–4×. We deliberately left it out of these measurements so that Run A and Run B remained comparable, and record it as the primary optimisation for future work. Reporting the measured bottleneck with its diagnosis is more useful than reporting a faster number with the cause hidden.

What this says about scalability. The honest conclusion is that this pipeline’s scale-out is bounded not by the algorithm but by two implementation properties: non-splittable input compression, which caps ingest parallelism at the number of files, and an uncached intermediate, which multiplies that capped work by four. Re-encoding the corpus as Parquet or splittable block-compressed files, and caching the intermediate, would be prerequisites to any meaningful study of scaling beyond two workers.

Cost. The entire cluster campaign — bring-up, debugging, and both full-corpus runs over a 7.78-hour cluster lifetime — cost \$5.01: \$4.39 in EC2/EMR compute, \$0.45 in S3 PUT requests, and \$0.17 in storage and EBS.

4.4 System Screenshots

TODO — action required before submission.

Insert screenshots here before submission. The EMR cluster `j-05114253MUVDDJMJS0Y5` is terminated but remains visible in the AWS console for roughly two months, so all of these can still be captured. Required: (1) EMR console cluster list showing the cluster and its `TERMINATED` state; (2) the cluster's Summary tab showing release label `emr-7.14.0`, applications, and instance counts; (3) the Hardware/Instance groups tab showing the `CORE` group resized from 1 to 2 `m5.xlarge`; (4) the Steps tab showing `one-worker-run-A` (`s-0276858I3LVNHH4ZD8H`) and `two-worker-run-B` (`s-09163011V7YC948GL1IZ`) both `COMPLETED` with their durations; (5) the S3 console showing `s3://csc7740-amazon-review/results/one-worker/` and `results/two-workers/` output prefixes; (6) the step stdout log showing the stage timing table and `PIPELINE COMPLETED SUCCESSFULLY` block; (7) CloudWatch graphs for `ContainerAllocated` and `YARNMemoryAvailablePercentage` across both runs.

5 Conclusion

What we accomplished. We built and executed a distributed Spark pipeline that processes 24.4 million Amazon reviews (17.1 GB raw JSON) end to end, joins them against 1.86 million product records, constructs an account co-review graph, detects communities with GraphFrames, measures review-text similarity with MinHash LSH, and ranks 443 candidate coordinated groups by a seven-signal score. The pipeline ran to completion on a single node and on a real two-configuration AWS EMR cluster, producing bit-identical analytical results in all three executions.

We went beyond building the detector to measuring it. The sensitivity sweep shows that the co-occurrence threshold moves the result by two orders of magnitude and that reported counts are meaningless without stated parameters. The injection test establishes 100% recall on rings inside the operating envelope with zero contamination of unrelated accounts. The scale-out experiment quantifies a $1.22\times$ speedup from doubling workers and explains precisely why it is not $2\times$.

Lessons learned. Three stand out. First, *the API boundary is a performance decision, not a style choice*: moving from RDDs with Python lambdas to DataFrame expressions converted a pipeline that could not finish into one that completes, because it removed the JVM-to-Python serialisation of every row. Second, *configuration defaults propagate in non-obvious ways*: a single hard-coded `fs.defaultFS=file:///` intended for local runs produced three unrelated-looking cluster failures, and we chased each symptom separately before finding the common cause. Third, *measure before optimising, and profile before explaining*: our initial assumption about which stage dominated was wrong, our first estimate of the remaining runtime was wrong by a factor of three, and only executor-log and CloudWatch evidence identified the real bottleneck as an uncached intermediate.

Future improvements. In priority order: (1) persist `member_reviews` and `enriched` to eliminate the four-fold recomputation, the single highest-value change; (2) re-encode the corpus as Parquet so ingest parallelism is not capped by gzip's non-splittability; (3) confirm the remote-shuffle-fetch hypothesis and, if borne out, co-locate partitions to reduce cross-node fetches; (4) extend to more categories and measure whether coordination density varies by product type; (5) replace the hand-chosen score weights with weights fitted against a labelled set, using the injection

harness to generate it; (6) add a streaming mode so new reviews can be scored incrementally rather than by full recomputation.

A Appendix 1: Source Code Summary and Engineering Contribution

TODO — action required before submission.

Each team member must verify the AI-assistance column for their own files before submission. The entries below describe the assistance actually used during development as recorded in this project’s working history.

File	Primary Functionality / Data Logic	AI Coding Assistant Usage Scope
AmazonReview CoordinationDF.py	Complete PySpark DataFrame pipeline: schema-explicit ingest, metadata join, adaptive time bucketing, co-occurrence self-join, GraphFrames connected components with label-propagation fallback, MinHash LSH text similarity, seven-signal scoring, same-account pair auditing	Extensive. Full DataFrame rewrite of an earlier RDD implementation was AI-assisted from a functional specification; schemas, scoring weights, pair-audit logic and the <code>WRITE_ENRICHED / fs.defaultFS</code> fixes were AI-authored and human-reviewed
emr/run_emr.sh	EMR step entry point: resolves the cluster’s real <code>fs.defaultFS</code> , pins staging / event-log / warehouse / checkpoint URIs to fully-qualified <code>hdfs://host:port</code> , creates the checkpoint directory, invokes <code>spark-submit</code> with GraphFrames	AI-authored during cluster debugging; each configuration fix derived from a specific observed failure
emr/bootstrap.sh	EMR bootstrap action installing NumPy, setuptools and GraphFrames on all nodes	AI-authored
analysis/ sensitivity.py	Time-window $\times$ co-occurrence-threshold parameter sweep, emitting pairs / accounts / components / largest-component per cell	AI-authored from a described experiment design
analysis/ injection.py	Synthetic coordination-ring injection and recall measurement, including foreign-account contamination check	AI-authored from a described experiment design

File	Primary Functionality / Data Logic	AI Coding Assistant Usage Scope
analysis/ count_spark_ operations.py	AST-based static counter of Spark call sites for Appendix 2	AI-authored
analysis/ create_sample_ dataset.py	Builds the <10 MB sample dataset for the deliverable package	[TODO: confirm]
analysis/ feasibility.py	Early sizing analysis: review volume and group-size distribution	[TODO: confirm]
analysis/ pairest.py	Co-occurrence pair-count estimation used to choose the threshold	[TODO: confirm]
analysis/ lockstep.py	Exploratory lockstep-behaviour probe	[TODO: confirm]
analysis/ nearDup.py	Exploratory near-duplicate review-text probe	[TODO: confirm]
analysis/ download_data.sh	Downloads the four review and four metadata archives from the McAuley Lab host	[TODO: confirm] (host URL corrected with AI assistance)
run_local.sh, run_all_ categories.sh	Local single-run and per-category sequential run drivers	[TODO: confirm]

B Appendix 2: Distributed Operations Count

Counts are static Spark call sites in the submitted Python source, obtained by AST analysis (analysis/count_spark_operations.py). A call site inside a loop is counted once. Local Python operations and Spark SQL expression helpers such as `F.col()`, `F.avg()` and `F.count()` are excluded, because they describe computation rather than executing it. Counted operations include `agg`, `approxSimilarityJoin`, `cache`, `coalesce`, `collect`, `connectedComponents`, `count`, `distinct`, `drop`, `dropDuplicates`, `filter`, `fit`, `groupBy`, `join`, `json`, `limit`, `localCheckpoint`, `orderBy`, `parquet`, `persist`, `repartition`, `select`, `show`, `take`, `transform`, `unionByName`, `unpersist`, `where` and `withColumn`.

File	# Distributed Operations
AmazonReviewCoordinationDF.py	151
analysis/sensitivity.py	28
analysis/injection.py	22
Total	201

A per-call-site audit listing every operation with its line number is included in the deliverable package as `analysis/DISTRIBUTED_OPERATIONS.md`.